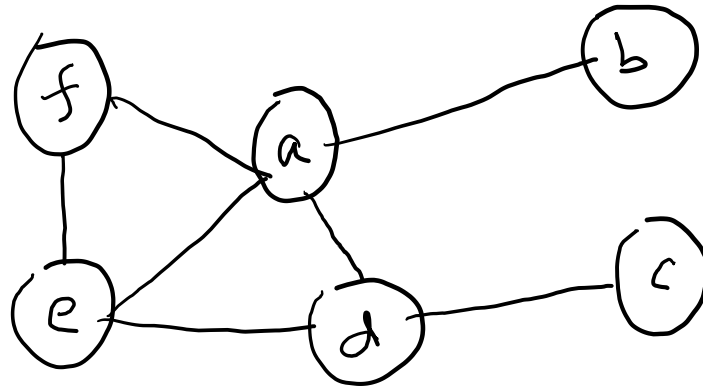

Introduction to Algorithm Engineering
Spring 2026
Lecture 5

Graphs – Biconnected Components

Problem Definition

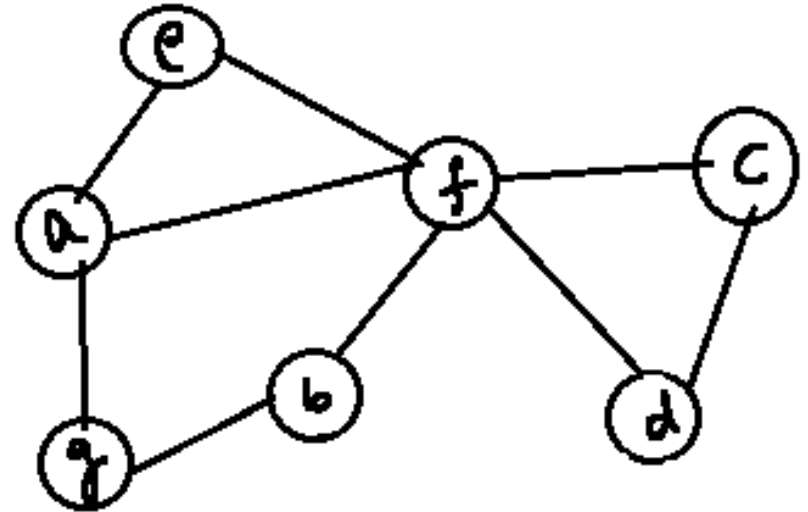
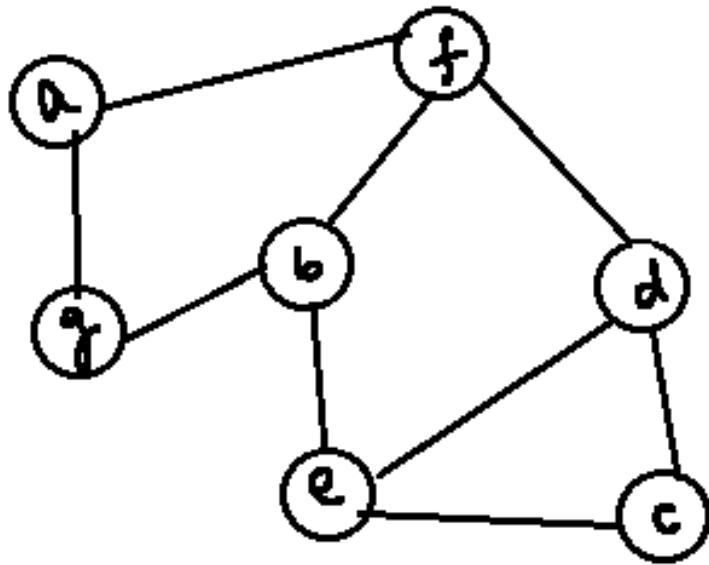


- We are given an undirected, graph $G = (V, E)$.
- We can define several notions of connectivity for G .
- G is connected –
 - G has only one connected component.
 - every pair of vertices u, v have a path between them.

Problem Definition

- Extend the notion of at least one path between every pair of vertices to
 - At least two paths between every pair of vertices.

Problem Definition



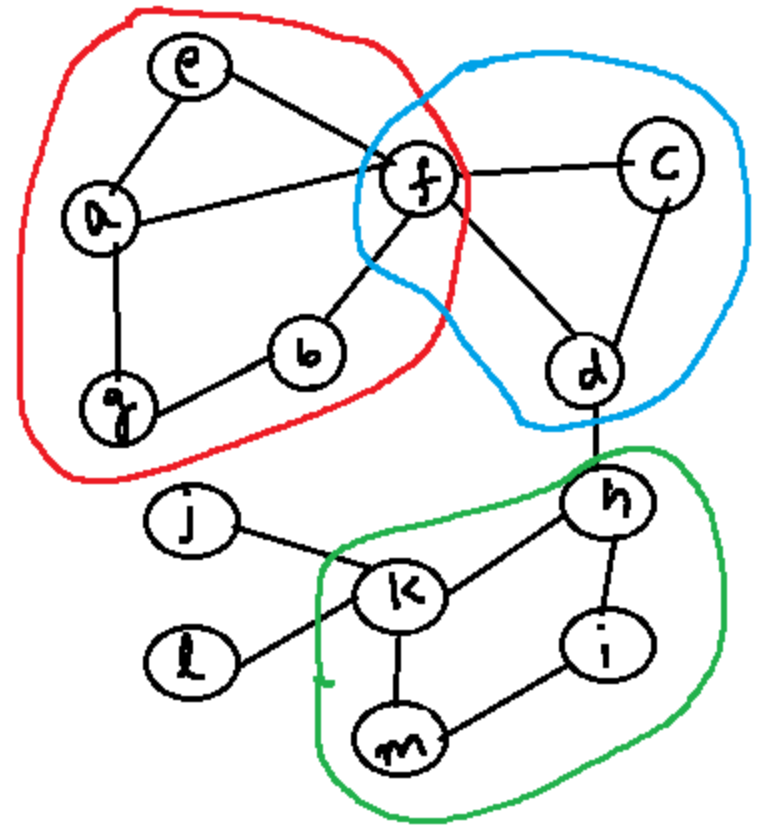
- A graph G where every pair of vertices have at least two vertex disjoint paths is called as **biconnected**, or **2-vertex connected**.
- If the paths are just edge disjoint, but not necessarily vertex disjoint, then the graph is said to be **2-edge connected**.

Problem Definition

- What if the graph G is not biconnected?
- Alike the question, what if G is not connected?
 - In this case, G has >1 connected components
 - Each connected component is a maximal set of vertices such that there is a path between every pair of vertices in the same connected component.

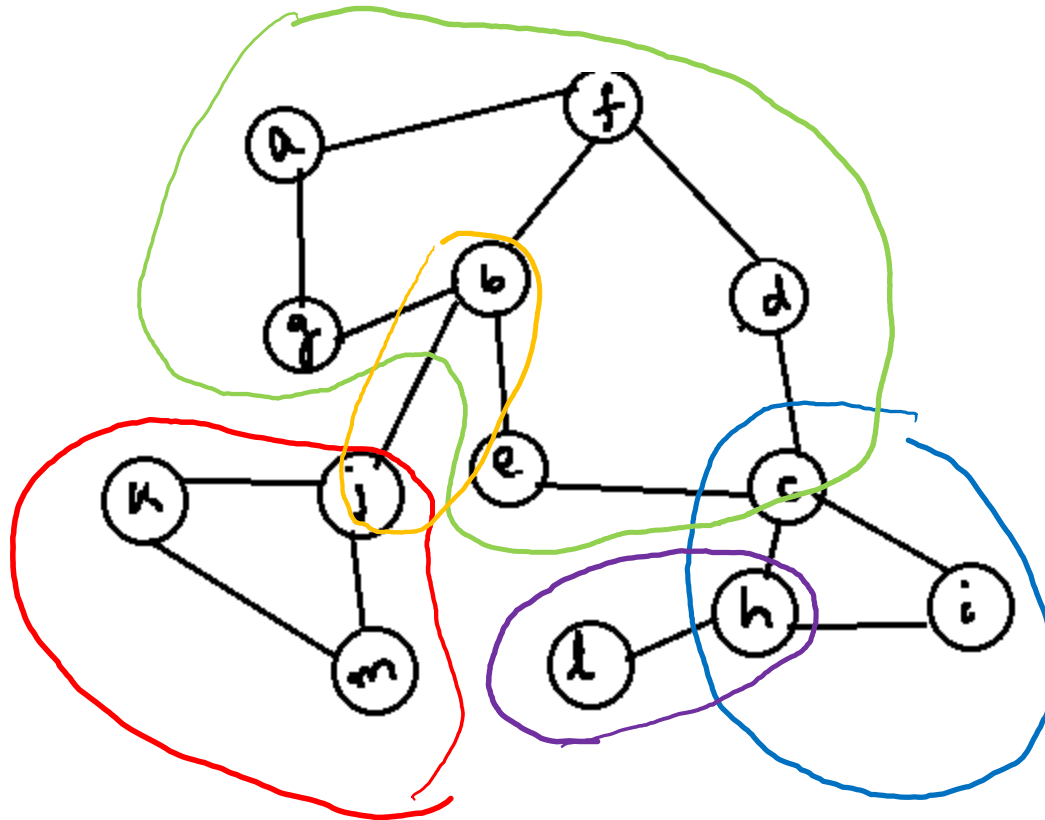
Problem Definition

- Extend such a notion to also 2-connectedness.
- Partition the **edges** of G into maximal subsets such that every pair of vertices in each subset have at least two vertex disjoint paths between them.
- Each such maximal subset that has at least two edges in it is called as a biconnected component of G .

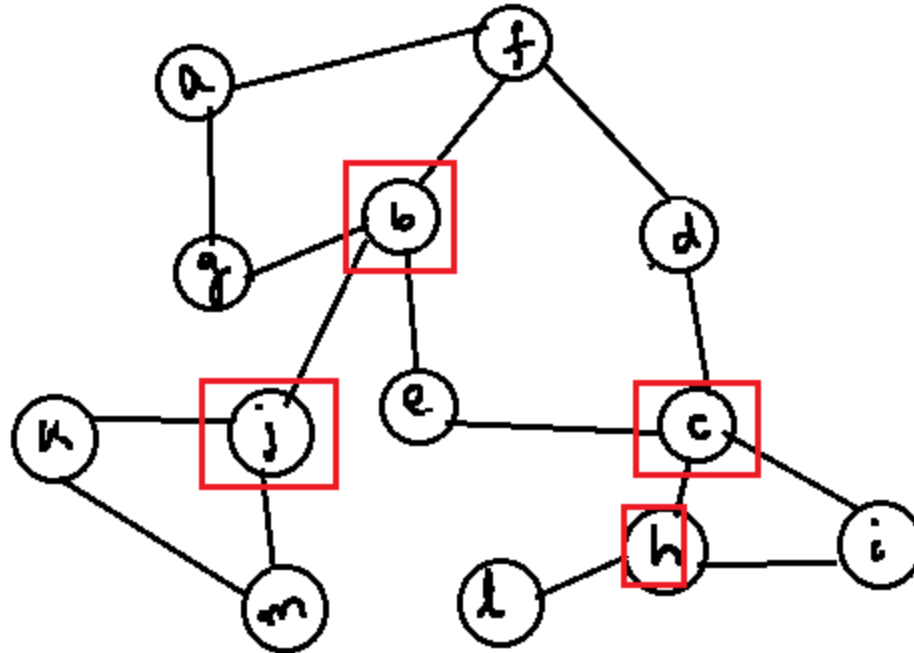


Example – Question

- Find the biconnected components of the graph below.

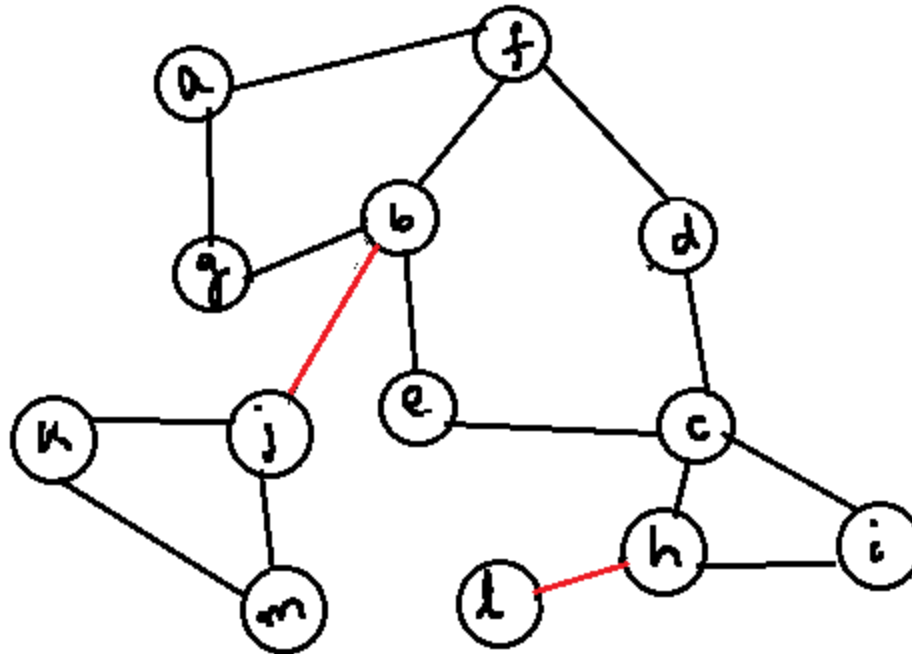


Articulation Points



- A vertex v in G is called as an **articulation point** if removing v disconnects G .
 - In other words, $G \setminus \{v\}$ has >1 connected component.

Bridges



- An edge in G is called as a **bridge** if removing e disconnects the graph G .
 - Again, $G \setminus \{e\}$ has >1 connected components.

Problem Definition

- There is a classical algorithm for identifying the biconnected components of a given undirected graph G .
- The algorithm is designed by Tarjan et al., and is entirely based on depth-first search.
- Read about this algorithm from the Cormen book.

Slota and Madduri Algorithm

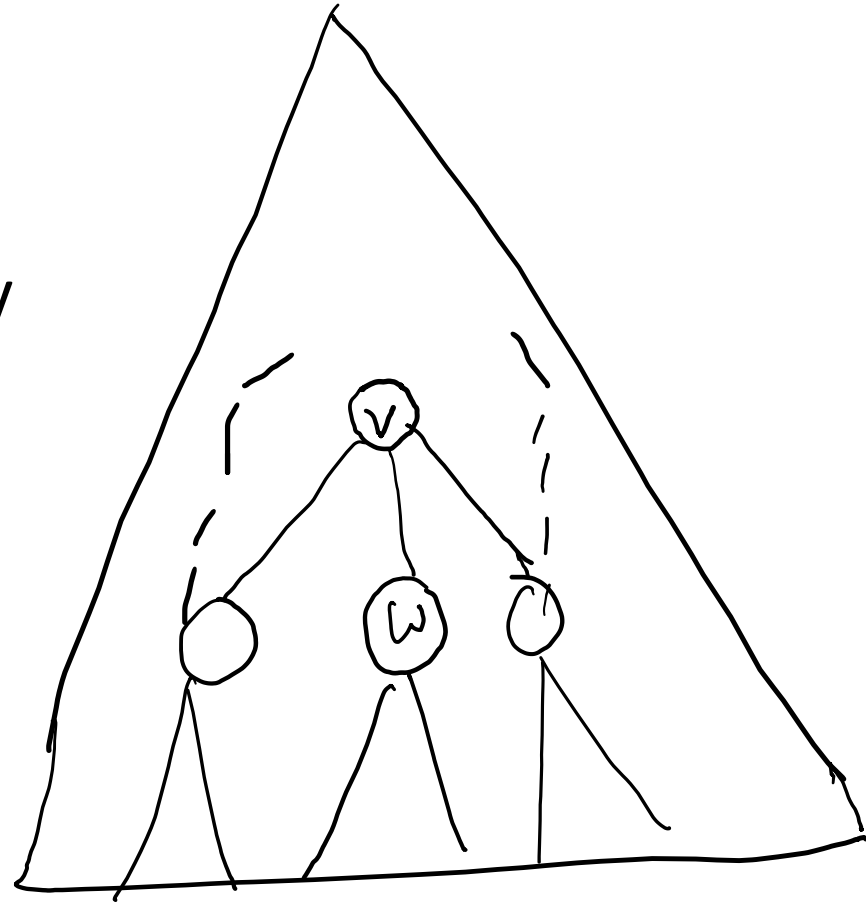
- Slota and Madduri (2014) proceed as follows.
 - Perform a BFS of G .
 - For all tree edges vw , with $v = p(w)$, remove the vertex v from G and perform another BFS from w .
 - If all such BFSs from w on $G \setminus \{v\}$ can reach to some vertex that is an ancestor of v , then v is not an articulation point.
- **+**: Notice that each BFS can be done in parallel without any dependencies on the other BFS.
- **–**: The downside is that one needs to perform n additional BFS traversals.

Slota and Madduri Algorithm

- Consider a BFS tree of G stored along with two other pieces of information: P and L
- The array P records the parent of each vertex w according to the BFS.
- The array L records the depth number of each vertex w . Root has depth number 0, its children have depth number 1, and so on.

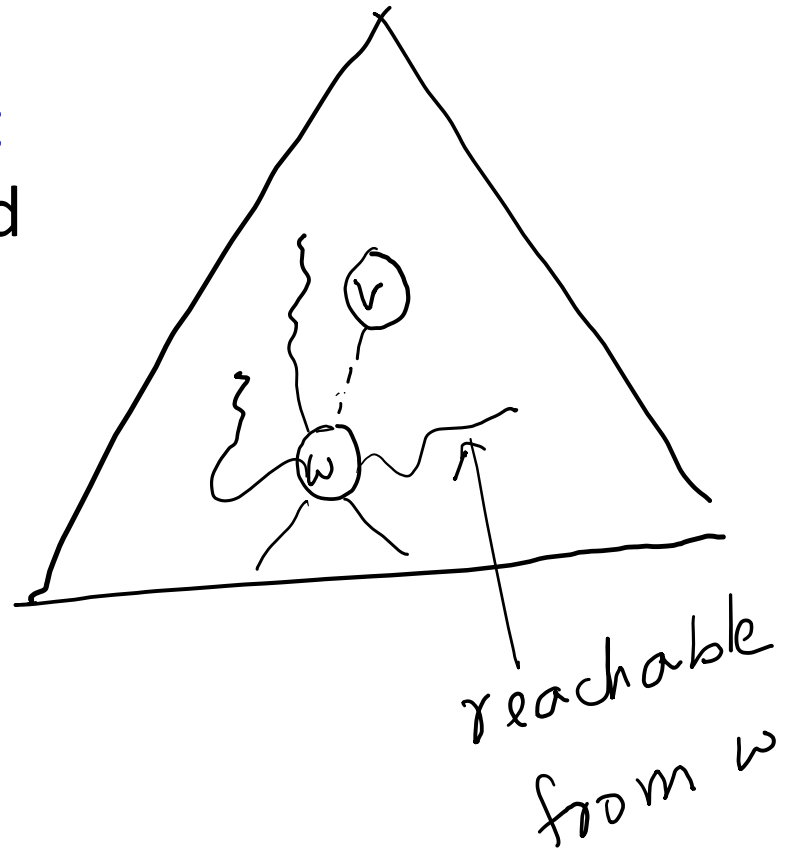
Slota and Madduri Algorithm

- **Claim:** A non-root vertex v in the BFS tree $\langle P, L \rangle$ is an articulation vertex if and only if it has at least one child w that cannot reach any vertex of **depth at most $L(v)$** when v is removed from G .
- Can be reworded as:



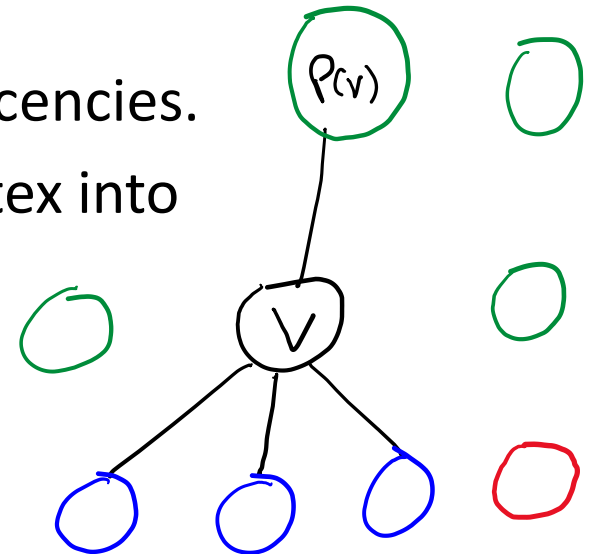
Slota and Madduri Algorithm

- **Claim:** A non-root vertex v in the BFS tree $\langle P, L \rangle$ is **not an** articulation vertex if and only if all its children w in the BFS tree ($P(w) = v$) can reach all other vertices in the graph G when v is removed from G .



A Claim

- **Claim:** A non-root vertex v in the BFS tree $\langle P, L \rangle$ is **not an** articulation vertex if and only if all its children w in the BFS tree ($P(w) = v$) can reach all other vertices in the graph G when v is removed from G .
- Proof: Consider a vertex v and all its adjacencies.
- Split the adjacencies of any non-root vertex into three disjoint subsets:
 - its children,
 - non child adjacencies at depth $L(v) + 1$,
 - and adjacencies at depth $L(v)$ or $L(v) - 1$.
- If v **were an** articulation point, then the adjacencies of v would be split up such that at least two of them lie in different connected components upon removal of v .

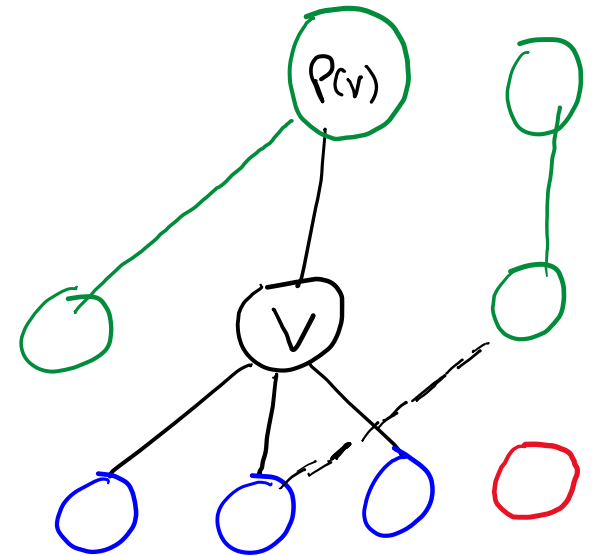


A Claim

- **Claim:** A non-root vertex v in the BFS tree $\langle P, L \rangle$ is **not an** articulation vertex if and only if all its children w in the BFS tree ($P(w) = v$) can reach all other vertices in the graph G when v is removed from G .
- Proof: Consider a vertex v and all its adjacencies.
- Split the adjacencies of any non-root vertex into three disjoint subsets:
 - its children
 - non child adjacencies at depth $L(v) + 1$, and
 - adjacencies at depth $L(v)$ or $L(v) - 1$.
- If v is **not an** articulation point, there must be an alternative path in the graph from adjacencies of v to every other vertex, and thus, between every pair of adjacencies of v as well.

A Claim

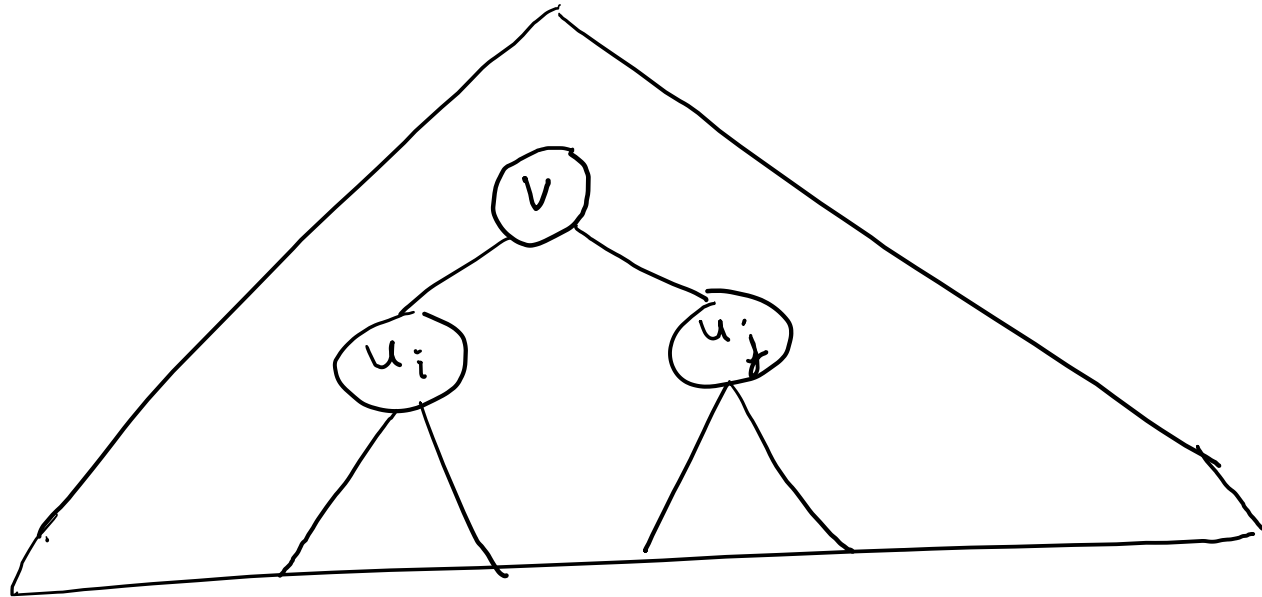
- To show that v is an articulation point, it suffices to check the reachability of the children of v in the BFS tree.
 - Is there such a child of v that cannot reach a vertex other than v at the same level as v .
 - Nonchild adjacencies of v have a path through their parent in the BFS tree to other vertices, and so we do not need to consider them explicitly.
 - If a child can reach some vertex at the same level as v , then it can reach all other vertices by tracing a path back to the root.



Slota and Madduri Algorithm

- **Claim:** A non-root vertex v in the BFS tree $\langle P, L \rangle$ is an articulation vertex if and only if it has at least one child w that cannot reach any vertex of depth at most $L(v)$ when v is removed from G .
- Can be reworded as:
- **Claim:** A non-root vertex v in the BFS tree $\langle P, L \rangle$ is **not an** articulation vertex if and only if all its children w in the BFS tree ($P(w) = v$) can reach all other vertices in the graph G when v is removed from G .
- Can be further reworded as:

Slota and Madduri Algorithm



- **Claim:** If a traversal from any $u_i \in V$ ($P(u_i) = v$) is not able to reach all other $u_j \in G$ ($P(u_j) = v$) when v is removed from the graph, then v is an articulation point.
- Further, if the only path in G between u_i and u_j requires v , then u_i and u_j are in separate biconnected components with v as an articulation point.
- We term v as the parent articulation vertex.

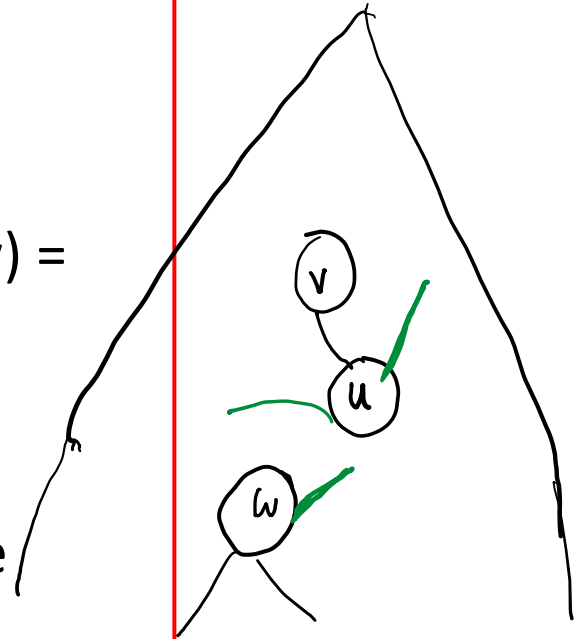
The Algorithm

Algorithm BFS-based algorithm to identify articulation points in BiCC decomposition.

1. procedure BFS-ARTPTS($G(V, E)$)
2. for all $v \in V$ do
 1. $\text{Art}(v) \leftarrow \text{false}$
 2. $\text{visited}(v) \leftarrow \text{false}$
3. Select a root vertex r
4. $P, L \leftarrow \text{BFS}(G, r)$
5. for all $u (\neq r) \in V$ where $P(u) \neq r$ do
 - $v \leftarrow P(u)$
 - if $\text{Art}(v) = \text{false}$ then
 - $\ell \leftarrow \text{BFS-L}(G, L, v, u, \text{visited})$
 - if $\ell \geq L(v)$ then
 - $\text{Art}(v) \leftarrow \text{true}$
6. Check if r is an articulation point

Slota and Madduri Algorithm

1. procedure BFS-L($G(V, E)$, L , v , u , visited)
2. Insert u into Q , Insert u, v into S
3. $\text{visited}(u) \leftarrow \text{true}$, $\text{visited}(v) \leftarrow \text{true}$
4. while $Q \neq \emptyset$ do
 - for all $x \in Q$ do
 - Remove x from Q
 - for all $\langle w, x \rangle \in E$ where $\text{visited}(w) = \text{false}$ do
 - if $L(w) < L(u)$ then
 - for all $s \in S$ do
 - $\text{visited}(s) \leftarrow \text{false}$
 - return $L(w)$
 - else
 - Insert w into Q
 - $\text{visited}(w) \leftarrow \text{true}$
5. return $L(u)$



Slota and Madduri Algorithm

- Different techniques proposed for handling the case of a root vertex.
- Solution 1: Pick a vertex x other than r , do a BFS in G from x , and see if r satisfies the conditions of the previous lemma.
 - Note that r is no longer the root of the BFS.
- Solution 2: In the same BFS from r , for each child u of r , check if all the siblings of u are reachable from u in the graph $G \setminus r$ by using another BFS from each such u .

Analysis

- A naive naive analysis indicates an upper bound of $O(nm)$ work.
- However, if we inspect vertices in level-synchronous order, and once a biconnected component is identified, mark all visited vertices as invalid.
- We also truncate BFS-L as soon as we encounter a vertex at the level of the parent.
- This limits the number of unnecessary traversals.
- The work performed is observed to be linear in practice.

More on this Problem

- This problem has attracted lot of attention in parallel computing in recent decade.
 - Fence Arbitrary Spanning Tree BCC [2023, Dhulipala et al.]
- We designed two other algorithms that are faster than the Slota and Madduri algorithm. [2015]
 - Our algorithm also works on GPUs very well. [2017]
- Applications to
 - Shortest paths [2017]
- More recently, we also explored
 - Sampling based biconnectivity to avoid BFS [2018]
 - Dynamic biconnectivity [2021]
 - Large graph biconnectivity [2026]